When Does Reconfiguration Awareness Pay Off? An Exact-Oracle Study of Learned Link Sleeping in Software-Defined Networks

Adeel Ahmad[a,*], Toqeer Ali Syed[a], Ali Akarma[a,b]

[a]*AI Center, Faculty of Computer and Information Systems, Islamic University of Madinah, Madinah, 42351, Saudi Arabia*
[b]*AI V&V Lab, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia*

Abstract

Learned controllers for energy-aware software-defined networks (SDNs) map each traffic matrix to a precomputed link-sleeping configuration and ignore the cost of changing configuration. We ask when that cost matters and how much of any gain needs learning. We formulate configuration over a finite library as a sequential problem that prices energy, reconfiguration, and wear, solve it exactly by dynamic programming, bound the library's loss with a per-interval integer program (at most 0.045%), and train LiRA, a history-conditioned imitation policy with a safety layer, as a strong learned controller. On the Abilene and GÉANT traces the optimal sequence is a single configuration, and snapshot learning, hysteresis, and LiRA reach the oracle cost to within 0.014%. On a semi-synthetic Nobel-Germany series, a gradient-boosted snapshot classifier flaps (11.7 changes per day) and costs 6.5% more than the oracle. Holding a configuration removes 97.7% of the changes at no energy loss, but tuned hysteresis, a rolling-horizon optimizer, and a static configuration with LiRA's fallback do the same: the static rule equals LiRA in 11 of 12 settings, and no difference survives a correction over all settings. The conclusions hold without the wear price and across noise variants, where the snapshot excess ranges from 0.0% to 32.5% with the noise's time correlation and scale. With the previous interval's traffic, costs barely change but the utilization bound is violated in up to 0.19% of the intervals. Reconfiguration awareness pays off only against flapping classifiers, and a rule that holds its configuration captures that benefit.

Keywords: Software-defined networking, energy efficiency, link sleeping, imitation learning, dynamic programming, reconfiguration cost, device lifetime

1. Introduction

Backbone and data-center links are provisioned for peak demand, yet their average utilization is far lower, so a large share of the power drawn by line cards and transceivers is spent carrying little or no traffic. Putting idle links to sleep was proposed more than two decades ago [1] and has since grown into a broad research area [2, 3]. Software-defined networking (SDN) makes the idea practical, because a logically centralized controller can observe the traffic matrix and switch link power states and forwarding rules network-wide [4, 5]. The underlying optimization is a mixed-integer problem that is NP-hard in general [6], and an exact solver needs up to 11.4 s per traffic matrix on the topologies we study (Section S8), a time that grows with the network and limits how often a controller can decide.

Recent work closes this speed gap with machine learning. A model is trained offline to map each incoming traffic matrix to one of a set of precomputed, energy-efficient configurations, which replaces the integer program at run time [7, 8]. The same pattern of replacing a solver with a learned mapping appears in traffic engineering [9, 10, 11, 12] and in combinatorial optimization more broadly [13].

A snapshot decision treats every interval as the first. The configuration chosen for the current traffic matrix ignores which configuration is installed, although moving between configurations has costs. Links must be woken or put to sleep, flow tables must be rewritten, and updates that are not carefully ordered can cause transient loops or loss [14, 15]. Each power-state transition also adds a thermal cycle that can offset the lifetime benefit of sleeping [16, 17]. The hypothesis is that snapshot classifiers flap between neighboring configurations when demand hovers near a decision boundary, and that a sequence-aware method should avoid this at little cost in energy.

This paper tests that hypothesis rather than assuming it. Its question is not whether a new controller beats old ones, but when the sequence matters at all, and how much of any gain needs learning. To answer it we need a reference that knows the best achievable sequence, a controller that can learn it, and baselines that hold a configuration without learning. We make four contributions.

- *Formulation and exact oracle.* We formulate energy-aware configuration as a sequential problem that jointly prices energy, reconfiguration, and wear. Over a finite library, the problem is a shortest path on a trellis, which dynamic programming solves exactly in $O(TK^2)$ time. We validate the solver against an independent mixed-integer program to 1×10^{-14} and prove that

*Corresponding author.
Email address: 443057803@stu.iu.edu.sa (Adeel Ahmad)

snapshot decisions can have regret that grows linearly with the horizon (Sections 3 and 4). A per-interval integer program bounds what the library restriction can cost (Section 6.3).

- *A representative sequence-aware learner.* LiRA (Lifetime- and Reconfiguration-Aware learning) imitates the oracle with cost-sensitive weights and dataset aggregation (DAgger) [18], conditions on the installed configuration and its dwell time, and wraps the policy in a feasibility safety layer (Sections 4 and S2). It serves as the strongest learned controller we could build from the oracle, so that a null result cannot be blamed on a weak learner. Drift detectors are described for completeness, and Section S10 reports that our drift study does not validate them.

- *An empirical answer to the question.* We evaluate on Abilene, GÉANT, and a semi-synthetic Nobel-Germany series under four load and power scenarios, against snapshot learning, hysteresis, minimum-dwell, rolling-horizon, static-configuration, integer-programming, genetic-algorithm, and greedy baselines. We separate the effect of the wear model by repeating the study without a wear price, and the effect of the synthetic noise by repeating it with temporally correlated and weaker noise. Ablations, decisions on one-interval-old traffic, a drift study, and rolling-origin folds complete the study (Sections 5 and 6).

- *Emulation of configuration changes.* A Mininet and Open vSwitch emulation replays the most active day of the Nobel-Germany series and measures rule-installation time, the duration of a configuration change, probe loss, and delay for each method (Section S12).

Our answer is a qualified one. On the two real traces, the energy-optimal configuration changes rarely: the exact oracle uses one configuration for the whole test period, snapshot classifiers change configuration at most 0.019 times per day, and LiRA, hysteresis, and the snapshot classifier reach the same energy savings (24.0% on Abilene and 29.8% on GÉANT) at costs within 0.014% of each other. LiRA therefore offers no measurable advantage there. On the Nobel-Germany series, whose demands are noisy, the snapshot classifier flaps (11.7 changes per day), and 99% of its changes save less than 0.1% of the interval energy. LiRA removes 97.7% of the changes, but in this scenario its trained policy is a constant classifier, and three rules without learning remove them as well: hysteresis tuned on validation data, a rolling-horizon optimizer with a persistence forecast, and a static configuration with LiRA's fallback. LiRA's learned component lowers the cost below that of the static rule in one scenario only, combined high load and high dynamic power. There its advantage over hysteresis is 0.09% and not significant, and the rolling-horizon optimizer is cheaper still. The same holds without the wear price and on three further noise variants of the Nobel-Germany series, in which the excess cost of the snapshot classifier ranges from 0.0% to 32.5%. The benefit of reconfiguration awareness therefore depends on the traffic and on the classifier, and in our data it comes from holding a configuration rather than from learning when to switch.

The remainder of the paper reviews related work (Section 2), formulates the problem (Section 3), presents LiRA (Sections 4 and S2), describes the setup (Section 5), reports results (Section 6), and discusses implications, limitations, and conclusions (Sections 7–9).

2. Related Work

Energy-aware networking puts idle links and devices to sleep [2, 3]. Integer programs and heuristics choose the links to sleep under capacity and connectivity constraints [6, 19, 20], and a recent study of operational ISP networks shows that the opportunity persists outside research backbones [21]. Learned controllers replace the optimizer: reinforcement learning has been used to put data-center links to sleep [22] and to trade delay against energy in SDN [23], and the works closest to ours map each traffic matrix to a precomputed configuration and report large speedups [7, 8]. We reimplement this snapshot principle as baseline B4 because we did not have access to the original code. Learned traffic engineering [9, 10, 12] also decides without pricing the change between configurations. FlexDATE [24] limits the disturbance of rerouting but does not put links to sleep.

Changing forwarding state is neither free nor instantaneous [14, 15], and power-state transitions affect device lifetime [16]. Online decisions with switching costs form metrical task systems [25]. Learning-augmented algorithms for them show that following a predictor blindly can be costly and that holding the current state is a robust fallback [26, 27]. Our setting is a metrical task system with a finite state space whose offline optimum is computable, which makes imitation learning with DAgger [18] natural, and our hysteresis and static baselines are simple state-holding rules. Section S1 of the Supplementary Material gives a full review.

3. Problem Formulation

Table S14 summarizes the notation.

3.1. Configuration Model

The network is a graph $G = (V, E)$ in which each undirected link e has capacity κ_e in each direction and consists of two ports that sleep and wake together. Time is divided into control intervals of length Δ, and D_t is the traffic matrix of interval t. A configuration $c = (x, r)$ consists of a link-state vector $x \in \{0, 1\}^{|E|}$, with $x_e = 1$ for an active link, and a routing r that splits each origin–destination demand over up to k precomputed paths that use only active links. A library $C = \{c_1, \ldots, c_K\}$ of such configurations is built offline (Section 4.1). The utilization $u_e(c, D)$ of link e in a direction is the offered load divided by κ_e, and c is feasible for D if its maximum link utilization (MLU) satisfies

$$\mathrm{MLU}(c, D) = \max_{e:\, x_e = 1,\ \mathrm{dir}} u_e(c, D) \le \theta \qquad (1)$$

for a utilization threshold $\theta < 1$.

3.2. Energy Model

Chassis stay powered, so the controllable power is that of link ports. Following load-dependent port profiles [28, 29, 30], the energy of c in interval t is

$$E(c, D_t) = \Delta \sum_{e \in E} \Big[x_e (2P^{\text{idle}} + P^{\text{dyn}}(u_e^{\rightarrow} + u_e^{\leftarrow})) + (1 - x_e)\, 2P^{\text{sl}} \Big], \tag{2}$$

where P^{idle}, P^{dyn}, and P^{sl} are the idle, full-load dynamic, and sleep power of one port, and $u_e^{\rightarrow}, u_e^{\leftarrow}$ are the directional utilizations. We report energy in units of one interval of the idle power of all links, so that the energy of the all-on configuration is close to one per interval.

3.3. Reconfiguration Cost

Moving from c' to c toggles $\|x - x'\|_1$ links, changes $\Delta F(c', c)$ forwarding entries (the number of switch and origin–destination pairs whose next hops differ), and moves a share $B(c', c) \in [0, 1]$ of the traffic to different paths, measured on the mean training traffic matrix so that R is a fixed table over library pairs. The reconfiguration cost is

$$R(c', c) = \tfrac{1}{3}\Big[\frac{\|x - x'\|_1}{\rho_1} + \frac{\Delta F(c', c)}{\rho_2} + \frac{B(c', c)}{\rho_3} \Big], \tag{3}$$

where ρ_1, ρ_2, ρ_3 are the 95th percentiles of the three terms over distinct library pairs, so that a typical large change costs about one unit.

3.4. Wear Model

We adopt the acceleration-factor view of Chiaraviglio *et al.* [16]: relative to an always-on device, sleeping lowers the failure rate, whereas power-state transitions raise it through thermal cycling. Let τ_e be the fraction of time link e sleeps and f_e its number of transitions per day. We use the linear form

$$\text{AF}_e = (1 - \tau_e) + \alpha\, \tau_e + \chi f_e = 1 - (1 - \alpha)\tau_e + \chi f_e, \tag{4}$$

where $\alpha < 1$ is the relative failure rate while asleep and χ weighs one transition per day. A value $\text{AF}_e > 1$ means that the link ages faster than an always-on link. Over a horizon of T intervals, $\sum_t W(c_{t-1}, c_t) = \frac{T}{n_d} \sum_e (\text{AF}_e - 1)$ for the additive per-step term

$$W(c', c) = \sum_{e \in E} \Big[\chi\, |x_e - x'_e| - \frac{1 - \alpha}{n_d} (1 - x_e) \Big]. \tag{5}$$

This decomposition is exact and makes wear additive over time. The parameters α and χ are assumed, representative values (Section S14), and we study the sensitivity of the results to them.

3.5. Sequential Objective

Given an initial configuration c_0, the sequential problem selects $c_1, \ldots, c_T \in C$ that minimize

$$J(c_{1:T}) = \sum_{t=1}^{T} \Big[E(c_t, D_t) + \lambda_R R(c_{t-1}, c_t) + \lambda_W W(c_{t-1}, c_t) \Big] \tag{6}$$

subject to (1) in every interval, where $\lambda_R, \lambda_W \geq 0$ set the price of reconfiguration and wear relative to energy. Snapshot methods instead use the per-interval rule

$$c_t^{\text{snap}} = \arg \min_{c \in C:\, \text{MLU}(c, D_t) \leq \theta} E(c, D_t), \tag{7}$$

which ignores the installed configuration. Online, the controller needs an estimate of D_t from flow counters or link loads [31], and Section 5.3 states what our evaluation assumes about it.

4. LiRA: Framework and Analysis

Fig. 1 shows the architecture. An offline plane builds the configuration library, computes sequence-optimal labels with the dynamic-programming oracle, and trains the policy. An online plane runs once per interval: it estimates the traffic matrix, builds features, queries the policy, checks feasibility, and installs only the difference between the current and the new configuration. Drift detectors connect the two planes.

4.1. Configuration Library

The library must be small enough for fast decisions and rich enough to contain an efficient feasible configuration for almost every traffic matrix. We cluster training traffic matrices with k-means (64 clusters) and take the element-wise 95th-percentile matrix of each cluster as its representative. For each representative we solve the path-based energy-minimization integer program with $k = 4$ shortest paths, link-pair sleeping, connectivity, and the bound (1), and we run NSGA-II [32] on port count and maximum utilization to add Pareto-diverse link-state vectors. For Abilene, whose link-state space is small, we also enumerate all connected link states. Duplicate vectors are merged, the routing of each vector is re-optimized by a linear program that minimizes maximum utilization, and further integer programs are solved for uncovered training and validation matrices until at least 99.5% are covered. The all-on configuration is always included. The library is pruned to K configurations by greedy energy coverage on every sixth training interval. Section 6.3 bounds what this restriction costs.

4.2. Exact Sequence-Optimal Oracle

Because all terms of (6) depend only on (c_{t-1}, c_t, D_t), the problem is a shortest path through a trellis whose stages are intervals and whose states are library members (Fig. 2). Infeasible states receive infinite cost. The backward recursion

$$\begin{aligned} V_{t-1}(c') &= \min_{c \in C} Q_t(c', c), \\ Q_t(c', c) &= E(c, D_t) + \lambda_R R(c', c) + \lambda_W W(c', c) + V_t(c) \end{aligned} \tag{8}$$

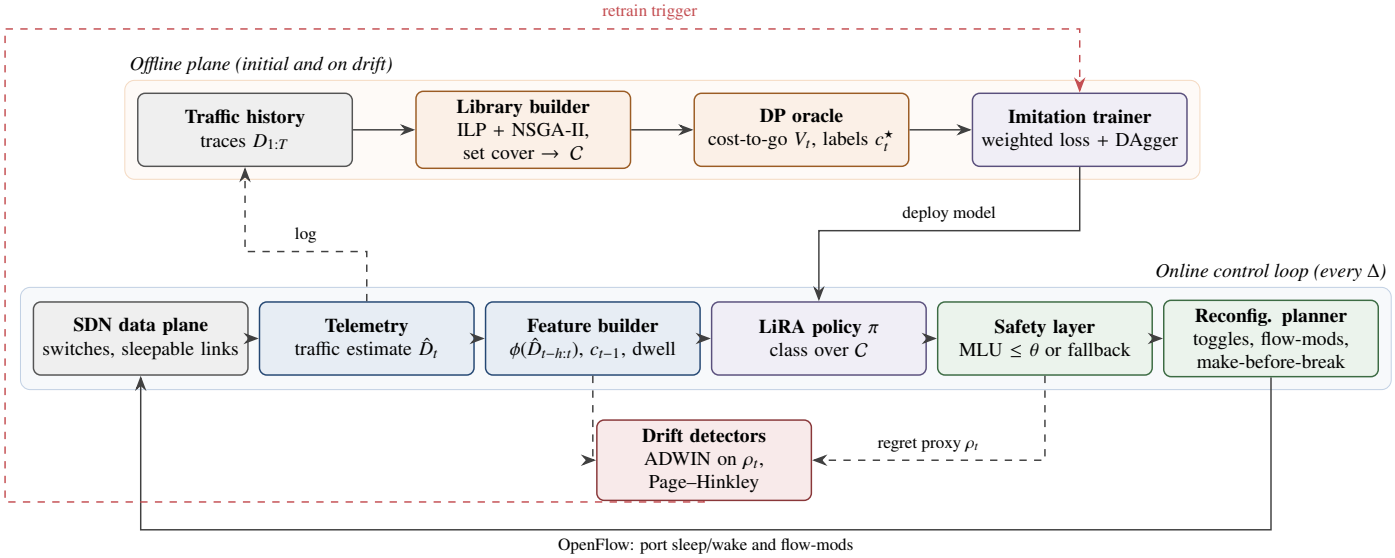


Figure 1: Architecture of LiRA. The offline plane builds the configuration library, computes sequence-optimal labels with the dynamic-programming oracle, and trains the policy. The online loop estimates the traffic matrix, selects a configuration conditioned on the installed one, enforces the utilization bound, and installs only the configuration difference. Drift detectors trigger retraining.

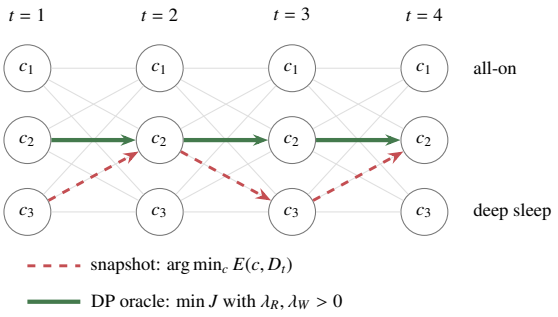


Figure 2: Configuration trellis (schematic). Nodes are library configurations per interval, and edges carry reconfiguration and wear costs. Snapshot selection flaps between c_2 and c_3 when traffic hovers near a decision boundary, whereas the oracle keeps c_2 when the extra energy is smaller than the switching cost.

computes the cost-to-go $V_t(c)$ of being in c at the end of interval t, with $V_T \equiv 0$. The oracle trajectory follows $c_t^\star = \arg\min_c Q_t(c_{t-1}^\star, c)$. The table $Q_t(c', \cdot)$ also gives the optimal action for *any* installed configuration c', which we use to relabel states visited by the learner.

Proposition 1. *For a fixed library C and known $D_{1:T}$, recursion* (8) *returns a feasible sequence minimizing* (6) *whenever one exists, in $O(TK^2)$ time and $O(TK)$ memory.*

Proof. By (5), the wear contribution is a sum of terms that each depend on one pair of consecutive configurations, and (2) and (3) depend on (c_{t-1}, c_t, D_t) only. Hence (6) is the length of a path through a layered graph with K nodes per layer, and edges into infeasible nodes have infinite weight. Bellman's principle of optimality yields (8), and each layer requires K^2 evaluations of precomputed quantities. □

4.3. History-Conditioned Policy

The policy maps the input s_t to a library index. The input contains the principal components (95% variance, fitted on training data) of the log-scaled origin–destination vector, the total volume, the shares of the ten largest demands, time of day and day of week, the changes of the principal components and the volume over the last 1, 3, and 6 intervals, the one-hot encoding of the installed configuration c_{t-1}, and its dwell time ℓ_{t-1} in intervals, capped at one day. The default model is a gradient-boosted tree classifier (XGBoost [33]), and Section S5 compares LightGBM [34], random forests [35], and a multilayer perceptron.

Cost-sensitive imitation. Plain classification treats all mistakes as equal, although choosing an action that is almost as cheap as the optimum is harmless. For each training state (t, c') we compute the oracle action $a^\star = \arg\min_c Q_t(c', c)$ and the cost gap $g_t(c') = \min_{c \neq a^\star} Q_t(c', c) - Q_t(c', a^\star)$ to the second-best action. The classifier is trained with the sample weight $w = \mathrm{clip}(g/\bar{g}, 0.2, 5)$, where $\bar{g}$ is the mean gap, so that decisions with a large cost difference matter more. This weighted cross-entropy loss is a practical surrogate for a cost-sensitive classifier.

DAgger. A policy trained only on oracle trajectories never sees the states that its own mistakes create. We apply DAgger [18] with three rounds: the current policy is rolled out on the training period, with the expert action used with probability $\beta_n = 0.5 \cdot 0.5^{n-1}$ in round n, the visited pairs (t, c_{t-1}) are relabeled from the stored Q tables without solving any new problem, and the policy is retrained on the aggregated data. Among the rounds, we keep the policy with the lowest sequence cost on the validation period. In 56 of our 60 main runs this is the policy of round zero, trained on oracle trajectories alone.

Remark. Because the oracle's table Q_t gives the optimal action from every installed configuration, the DAgger analysis

for task costs [18] applies with a per-deviation loss of at most $u = \max_{t,c',c}[Q_t(c', c) - \min_{\tilde{c}} Q_t(c', \tilde{c})]$, so the cost gap grows linearly in the learner's disagreement with the oracle rather than quadratically in T. Our weighted cross-entropy only approximates the cost-sensitive loss of that analysis, so this motivates the design and is not a guarantee for the implemented learner.

4.4. Safety Layer

At run time, the configuration chosen by the policy is checked against (1) using the traffic estimate. If it is infeasible, the controller falls back to

$$c_t = \operatorname*{arg\,min}_{c \in C:\, \mathrm{MLU}(c, \hat{D}_t) \le \theta} \Big[E(c, \hat{D}_t) + \lambda_R R(c_{t-1}, c) + \lambda_W W(c_{t-1}, c) \Big], \tag{9}$$

which is evaluated for all K configurations. If no configuration is feasible, the all-on configuration is used and the interval is counted as a violation.

Proposition 2. *If the all-on configuration satisfies* MLU $\le \theta$ *for* $\hat{D}_t$*, the safety layer returns a configuration that satisfies* (1) *for* $\hat{D}_t$*.*

Proof. The all-on configuration belongs to C by construction, so the feasible set in (9) is non-empty whenever the policy's choice is infeasible. □

The guarantee holds for the estimate $\hat{D}_t$, not for the traffic that then arrives. Section 5.3 states which estimate our evaluation uses, and Section S16 measures the violations when the estimate is one interval old.

5. Experimental Setup

5.1. Datasets and Scenarios

Table S15 lists the data. Abilene (Zhang *et al.* [31]) and GÉANT (Uhlig *et al.* [36]) topologies and traffic matrices come from SNDlib [37]. SNDlib offers only one day of five-minute matrices for Nobel-Germany, while day-of-week features and a chronological split need several weeks. We therefore generate a 30-day semi-synthetic series: a gravity model fitted to the SNDlib day is modulated by the weekly profile of Abilene and multiplied by independent log-normal noise per origin–destination pair and interval, whose scale ($\sigma = 1.23$) is fitted to the residuals of Abilene. This noise has no temporal correlation and is therefore harsher than the noise of a real trace.

We removed unfilled gaps and all-zero measurement outages, interpolated gaps of up to three intervals, removed eleven GÉANT intervals whose total volume exceeds ten times its one-day rolling median, and split each series chronologically into 60% training, 15% validation, and 25% test. Sequences never cross an unfilled gap. Traffic is scaled so that the 99.9th percentile of the all-on maximum link utilization on the training split is 0.5, which makes the scenario "base".

We add three scenarios to probe regimes in which sequence decisions could matter more: *high load* (demands ×1.6, a training peak of 0.8), *high dynamic power* (the load-proportional port power tripled to 15 W), and *stress* (both). The library is rebuilt for each scenario.

5.2. Baselines

B0 keeps all links on and routes each demand on its shortest path. B4 is our reimplementation of the snapshot-learning principle [7, 8]: an XGBoost classifier that maps the current traffic features to the energy-optimal feasible library member. If its choice violates θ, B4 falls back to the feasible configuration with the lowest energy. B6 applies hysteresis to B4, switching only if the energy gain of the proposed configuration exceeds a threshold, and B7 applies a minimum dwell time. Both thresholds are tuned on the validation split by sequence cost J. B2 is a rolling-horizon controller that solves the sequential problem exactly over the library for the next 12 intervals with a persistence forecast and applies the first action. It needs no learning and is restricted to the library. The static rule holds the configuration that the oracle uses most often on the training period and applies LiRA's fallback (9) whenever that configuration is infeasible. It is LiRA with its classifier replaced by a constant. B1 solves the energy integer program for each traffic matrix with HiGHS [38], B3 runs NSGA-II per traffic matrix, and B5 is an ElasticTree-style greedy heuristic [20]. B1, B3, and B5 are evaluated on a subsample of 60 test intervals (15 for B3 on Abilene) because of their run time. We also report the offline oracle O*, which uses the future and is a lower bound for every controller restricted to the library. Section 6.3 bounds controllers that are not. A deep reinforcement learning baseline is not included.

5.3. Information Available to the Controller

Unless stated otherwise, every online method chooses the configuration of interval t with the realized matrix D_t: its features are computed from D_t, and its safety check uses $\mathrm{MLU}(c, D_t)$. This idealizes the estimate $\hat{D}_t$ of Algorithm S2, and it gives the safety layer the information that the oracle has. Section S16 repeats the comparison when every online method decides with D_{t-1}, the last completed measurement, with and without a safety margin of 0.05 on θ.

5.4. Metrics and Prices

Energy savings S are measured relative to B0. We report configuration changes and link power-state transitions per day, flow-modification messages, the share of intervals with a violation of θ, the sequence cost J of (6), the mean and maximum acceleration factor, and the decision latency measured with batch size one. The retention $X = S_{\mathrm{LiRA}}/S_{\mathrm{B4}}$ and the reductions $Y = 1 - N^{\mathrm{ch}}_{\mathrm{LiRA}}/N^{\mathrm{ch}}_{\mathrm{B4}}$ and $Z = 1 - N^{\mathrm{tr}}_{\mathrm{LiRA}}/N^{\mathrm{tr}}_{\mathrm{B4}}$ are defined only when B4 changes configuration. The reported cost J is (6) without the constraint: an interval that violates θ adds no penalty and is counted in the violation share instead, and the first test interval and the first interval after a measurement gap carry no switching cost. J also includes the modeled wear credit of sleeping links, so it is not an energy figure: absolute values of J are lower than the energy of the always-on network, and only differences between methods are meaningful. With the default prices a sleeping link earns a wear credit of 0.040 per interval, while sleeping it saves 0.060, 0.035, and 0.025 of the normalized energy on Abilene, Nobel-Germany, and GÉANT,

so the wear model rewards sleeping about as much as the energy model does.

The prices are set from operating considerations. One typical large change costs 0.2% of the daily energy of the all-on network, so $\lambda_R = 0.002\, n_d$, and one link transition costs 0.08% of it, so $\lambda_W = 0.0008\, n_d/\chi$, where n_d is the number of intervals per day. For stochastic components (classifier seeds and DAgger rollouts) we use five seeds and report means with 95% confidence intervals. Where a policy is constant, all seeds give the same result and the confidence interval has zero width. We compare the per-day sequence costs of LiRA with those of B4, B6, and B7 with the Wilcoxon signed-rank test and correct the three comparisons per setting with the Holm method. Because twelve settings are tested, we also apply the Holm correction to all 36 comparisons at once and report the result. Confidence intervals that round to zero, which occur when every seed gives the same sequence, are not printed.

6. Results

Two facts about the trained LiRA policies frame all results. In 30 of the 60 main runs (all Abilene runs and the Nobel-Germany base and high-dynamic-power scenarios) the trained policy is a constant classifier, because the oracle uses a single configuration on the training period. LiRA then holds the configuration that was optimal on the training period and switches only through its fallback, and its ablations are trivial. In 56 of the 60 runs the policy selected on validation data is that of round zero, before any DAgger data. The static rule of Section 5 makes the consequence measurable: it reaches the same cost as LiRA in 11 of the 12 settings of Table 2.

6.1. Do Real Traces Need Sequence Awareness?

Table 1 gives the main comparison in the base scenario. On Abilene and GÉANT, B4, B6, B7, LiRA, and the oracle reach the same energy savings (24.0% and 29.8%), make at most 0.019 configuration changes per day, and have sequence costs within 0.014% of each other. The rolling-horizon controller B2, which relies on a persistence forecast, is marginally worse on Abilene. The oracle itself uses a single configuration for the whole test period (41.8 and 28.1 days). B4 never changes configuration on GÉANT, so the reductions Y and Z are undefined there, and on Abilene they would rest on less than one change on average over the test period. LiRA is no worse and no better. The per-day cost differences between LiRA and B4, B6, and B7 are not significant on these traces (Table 2).

The Nobel-Germany series behaves differently. B4 changes configuration 11.7 times per day and makes 23.4 link transitions per day, whereas the exact snapshot rule (7) would change 63.7 times per day. LiRA makes 0.27 changes per day, a reduction of $Y = 97.7\%$ and $Z = 97.7\%$ at an energy retention of $X = 100.0\%$, and the mean acceleration factor drops from 0.855 for B4 to 0.846. The reduction does not come from learning. The policy is constant in this scenario and holds one configuration in 2 159 of the 2 160 test intervals, and the static rule gives the same result. Hysteresis (B6) and the rolling-horizon B2 both make 0.13 changes per day at a cost of 875.3, slightly below LiRA's 875.9, so LiRA is not better than either. The paired test gives an adjusted $p = 0.88$ against B6, with a mean daily cost difference (LiRA minus B6) of 0.07 and LiRA cheaper on 38% of the days, and $p = 0.023$ against B4. These reductions are relative to a gradient-boosted B4 that flaps heavily. A random-forest B4 changes configuration only 0.5 times per day (Section S5), so the size of Y and Z depends on the classifier that we reimplemented. The test uses eight daily samples, the last of them a half day, so the smallest attainable two-sided p-value is 0.0078. With the Holm correction over all 36 comparisons of Table 2, 0 remain significant (smallest adjusted $p = 0.28$), including the comparison with B4. The cost differences we report are therefore descriptive. Their consistency across seeds, folds (Section S11), and noise variants (Section S15) carries more weight than any single test.

Fig. S4 shows that the flapping is harmless to remove. Among the 96 configuration changes of B4 (seed 0) in the test period, none saves more than 0.41% of the interval energy, 99% save less than 0.1%, and the median saving is below 0.01%. Such changes are expected when demand hovers near the boundary between two nearly equivalent configurations, and avoiding them costs essentially no energy.

6.2. Other Scenarios and Where the Differences Come From

Table 2 summarizes all topologies and scenarios. Higher load and higher dynamic power change nothing on Abilene and GÉANT: the oracle remains a single configuration. On Nobel-Germany, higher load makes the optimal sequence non-constant (0.80 changes per day). B4 then makes 1.76 changes per day, B6 makes 1.07, and LiRA makes 0.80. LiRA's sequence equals the oracle's in every test interval and every seed (0 differing intervals), at a cost of 880.9 against 882.2 for B2, 887.0 for B6, and 890.7 for B4. The match has a simple cause. The expert is nearly constant, LiRA predicts the default configuration, and its safety layer, which sees the realized matrix, switches in exactly the intervals in which the default becomes infeasible, as the oracle does. The static rule reaches the same cost (880.9), and so does every ablation that keeps the safety layer (Section S6). Over 7.5 days of test data the difference to B6 is not significant ($p = 0.31$, LiRA cheaper on 50% of the days). Under stress, the only setting in which LiRA differs from the static rule, LiRA (898.6) is cheaper in total than B6 (899.4) and the static rule (902.0), but B2 (895.4) comes closer to the oracle (894.3). The difference to B6 is 0.09% of the cost, LiRA is cheaper on only 25% of the days, and the test gives $p = 0.64$.

Fig. S2 shows the busiest test week of the Nobel-Germany series. B4 changes configuration 88 times during the week, whereas LiRA makes only 2 changes. Fig. S3 plots total cost against configuration changes for the hysteresis and dwell-time sweeps and for the price sweep of LiRA and the oracle. LiRA and the oracle lie at the low-cost, low-change end of the plotted points, and a tuned hysteresis threshold reaches nearly the same point.

Table 1: Main Comparison in the Base Scenario (Test Split). Means with 95% Confidence Intervals over Five Seeds for B4, B6, B7, and LiRA; Single Values Otherwise. Latency Is the Median Decision Time

Method	S (%)	Changes/d	Transitions/d	MLU viol. (%)	J	$\overline{\text{AF}}$	Latency (ms)
Abilene							
B0 always-on	0.00	0.00	0.00	0.00	12037.5	1.000	–
B4 snapshot ML	23.98	0.02±0.05	0.08±0.21	0.00	7227.7±2.8	0.867	0.989
B6 B4+hysteresis	23.98	0.00	0.00	0.00	7226.7	0.867	0.989
B7 B4+min-dwell	23.98	0.02±0.05	0.08±0.21	0.00	7227.7±2.8	0.867	0.989
B2 rolling horizon	23.95	0.00	0.00	0.00	7230.4	0.867	0.433
LiRA	23.98	0.00	0.00	0.00	7226.7	0.867	0.008
Oracle O*	23.98	0.00	0.00	0.00	7226.7	0.867	–
GÉANT							
B0 always-on	0.00	0.00	0.00	0.00	2705.2	1.000	–
B4 snapshot ML	29.85	0.00	0.00	0.00	603.7	0.833	1.599
B6 B4+hysteresis	29.85	0.00	0.00	0.00	603.7	0.833	1.599
B7 B4+min-dwell	29.85	0.00	0.00	0.00	603.7	0.833	1.599
B2 rolling horizon	29.85	0.00	0.00	0.00	603.7	0.833	0.432
LiRA	29.85	0.00	0.00	0.00	603.7	0.833	1.571
Oracle O*	29.85	0.00	0.00	0.00	603.7	0.833	–
Nobel-Germany							
B0 always-on	0.00	0.00	0.00	0.05	2163.7	1.000	–
B4 snapshot ML	27.63	11.68±0.89	23.36±1.78	0.00	931.7±4.3	0.855±0.001	0.949
B6 B4+hysteresis	27.63	0.13	0.27	0.00	875.3	0.846	0.949
B7 B4+min-dwell	27.63	2.29±0.18	4.59±0.36	0.00	885.8±0.9	0.848	0.949
B2 rolling horizon	27.63	0.13	0.27	0.00	875.3	0.846	0.423
LiRA	27.63	0.27	0.53	0.00	875.9	0.846	0.008
Oracle O*	27.63	0.00	0.00	0.00	874.7	0.846	–

6.3. What the Library Gives Up

Because the switching terms of (6) are non-negative, the sum over the test intervals of the optimal single-interval cost, with free link states and free routing over the k paths, is a lower bound on the cost J of any controller, with or without a library, including the optimum of the full multi-period mixed-integer program. We solve this single-interval integer program with HiGHS for all 21 203 test intervals of five settings (median 214 ms per interval, largest relative gap 9×10^{-7}) and use the solver's dual bound. Table 3 compares the bound with the library snapshot optimum, which picks the best library member in every interval and also ignores switching, with the oracle, and with LiRA. The library restriction itself costs at most 0.045% in any setting. The oracle lies within 0.55% of the bound, and on Nobel-Germany under high load and stress most of that gap is the cost of holding a configuration instead of re-optimizing every interval, which the bound ignores. A multi-period program without a library can therefore improve on the oracle by at most 0.55%.

6.4. Robustness to Noise Structure and Stale Traffic

Two further experiments, given in full in Sections S15 and S16 of the Supplementary Material, test the assumptions behind these results. First, we regenerate the Nobel-Germany series with time-correlated noise (AR(1), $\rho = 0.5$ and $\rho = 0.9$) and with half the noise scale, and retrain every method. With $\rho = 0.5$ the snapshot classifier changes configuration 57.8 times per day and costs 32.5% more than the oracle, with $\rho = 0.9$ only 0.6%, and with half the noise it does not flap. In every variant B6, B2, the static rule, and LiRA stay within 0.36% of the oracle, and LiRA costs the same as the static rule. Second, when every online method decides with the previous interval's traffic D_{t-1}, costs move by at most 0.002%, but the safety layer no longer guarantees θ: up to 0.19% of the Nobel-Germany intervals violate it, at single-interval traffic peaks that a margin of 0.05 does not remove. LiRA's exact match with the oracle under high load therefore depends on knowing the traffic of the interval it configures.

7. Discussion

When does sequence awareness matter? In our data, it matters when the snapshot decision flaps. This happens with noisy demands whose fluctuations persist for a few intervals (Section S15) and with some classifiers (XGBoost, LightGBM, and the perceptron, but not the random forest), and it does not happen on smooth real traces. The wear model does not change this (Section S9). Where flapping occurs, holding a configuration removes nearly all changes without losing energy, and no learning is needed to do so: tuned hysteresis, a rolling-horizon optimizer with a persistence forecast, and a static configuration with a fallback all match LiRA.

What the oracle and LiRA add. The oracle makes the cost of any controller measurable on a trace. Together with the per-interval bound, it shows that the library loses at most 0.045% and that the real traces are nearly constant. LiRA's learned component changes the outcome in one setting, the stress scenario, where its total cost is 0.09% below that of hysteresis ($p = 0.64$) and above that of the rolling-horizon optimizer. Its exact match with the oracle under high load comes from the safety layer and from knowing the current traffic. With one-interval-old traffic

Table 2: All Topologies and Scenarios: Configuration Changes per Day, Total Cost J, and Holm-Adjusted p-Values of the Wilcoxon Test on Per-Day Cost (B4, B6, and LiRA: Mean over Five Seeds)

		Changes/d				Total cost J					p_{Holm}	
Topology	Scenario	B4	B6	B2	LiRA	B4	B6	B2	LiRA	O*	vs B4	vs B6
Abilene	base	0.02	0.00	0.00	0.00	7227.7	7226.7	7230.4	7226.7	7226.7	0.539	1.000
Abilene	high load	0.00	0.00	0.02	0.00	7233.0	7233.0	7237.2	7233.0	7233.0	1.000	1.000
Abilene	high dynamic power	0.02	0.00	0.00	0.00	7248.7	7247.7	7258.7	7247.7	7247.7	0.539	1.000
Abilene	stress	0.00	0.00	0.02	0.00	7266.6	7266.6	7277.7	7266.6	7266.6	1.000	1.000
GÉANT	base	0.00	0.00	0.00	0.00	603.7	603.7	603.7	603.7	603.7	1.000	1.000
GÉANT	high load	0.00	0.00	0.00	0.00	610.4	610.4	610.4	610.4	610.4	1.000	1.000
GÉANT	high dynamic power	0.00	0.00	0.00	0.00	625.0	625.0	625.0	625.0	625.0	1.000	1.000
GÉANT	stress	0.00	0.00	0.00	0.00	644.8	644.8	644.8	644.8	644.8	1.000	1.000
Nobel-Germany	base	11.68	0.13	0.13	0.27	931.7	875.3	875.3	875.9	874.7	0.023	0.875
Nobel-Germany	high load	1.76	1.07	0.93	0.80	890.7	887.0	882.2	880.9	880.9	0.117	0.312
Nobel-Germany	high dynamic power	11.68	0.27	0.13	0.27	939.5	883.7	883.2	883.7	882.9	0.023	1.000
Nobel-Germany	stress	1.76	1.07	0.93	0.91	903.1	899.4	895.4	898.6	894.3	0.164	0.641

Table 3: Lower Bound on the Cost of Any Controller (Sum of Per-Interval Optima), the Library Snapshot Optimum, the Oracle, and LiRA on the Test Split. Library Loss Is the Snapshot Optimum Above the Bound, O* Gap the Oracle Above the Bound, Both Relative to the Oracle Cost

Setting	Bound	Lib. snapshot	O*	LiRA	Lib. loss (%)	O* gap (%)
Abilene	7226.7	7226.7	7226.7	7226.7	0.000	0.000
GÉANT	603.7	603.7	603.7	603.7	0.011	0.011
Nobel, base	874.5	874.5	874.7	875.9	0.001	0.024
Nobel, high load	876.9	877.2	880.9	880.9	0.043	0.463
Nobel, stress	889.3	889.7	894.3	898.6	0.045	0.554

LiRA keeps the oracle's cost but violates the utilization bound in 0.14% of the intervals.

Relation to learning-augmented algorithms. Our results match a lesson of online algorithms with predictions [39, 26]: a controller that follows a predictor blindly can pay far more than the optimum, and combining the predictor with a rule that holds its state recovers nearly all of the loss. In our setting the holding rule alone already suffices, because the oracle itself rarely switches.

Choosing the prices. The prices λ_R and λ_W are operator policy. The Pareto plots (Fig. S3) show how much cost a setting gives up, and because the oracle runs in seconds (Section S8), an operator can explore prices offline before deploying a policy.

Deployment. The safety layer and the all-on fallback are independent of the model, so a mispredicting model degrades to higher energy use and not to congestion, as long as the all-on configuration is feasible and the traffic estimate is accurate (Proposition 2). With one-interval-old estimates the layer no longer prevents violations on noisy traffic (Section S16), and a fixed margin does not help against single-interval peaks. A deployment therefore needs either a traffic forecast or capacity headroom sized to the peaks, which we did not evaluate. The fallback rate is also a monitoring signal for a stale library.

8. Limitations

The flapping series is semi-synthetic, the two real traces are old research backbones, and the Nobel-Germany test split covers 7.5 days, so per-day tests have little power and no comparison survives a correction over all settings. Port power and wear parameters are assumed, B4 reimplements the snapshot principle rather than a published system, and no deep reinforcement learning baseline was trained. The main results let each method see the traffic it configures, and the drift study did not validate the detectors. The learned component matters in one scenario only: the trained policy is constant in 30 of 60 runs, and the static rule equals LiRA in 11 of 12 settings. Section S3 of the Supplementary Material lists all threats to validity.

9. Conclusion

We formulated energy-aware SDN configuration as a sequential problem, solved it exactly over a configuration library, bounded what the library gives up, and used the solution to train a history-conditioned policy with a safety layer. The empirical result is a qualified one. On two real traces, a snapshot classifier is already within 0.014% of the oracle, and the oracle and LiRA hold a single configuration. On noisy semi-synthetic series, a snapshot classifier flaps and costs up to 32.5% more than the oracle, and holding a configuration removes nearly all of that excess at no loss of energy. This holds without the wear price and for every noise variant we tried. Tuned hysteresis, a rolling-horizon optimizer, and a static configuration with a fallback do the same, and LiRA's learned component changes the result in one scenario only, by an amount that is not significant. The safety layer guarantees the utilization bound only when the traffic of the configured interval is known. We release the oracle, the baselines, and all scripts (`https://github.com/adeliusa486/-lira-sdn-energy`) so that other traces and power models can be tested. The extensions we see are traffic with realistic temporal structure, measured router power, controllers that forecast traffic, and hardware with sleep-capable line cards.

Data availability

The traffic matrices are public: Abilene, GÉANT, and the one-day Nobel-Germany series come from SNDlib [37, 36, 31],

and the 30-day Nobel-Germany series is generated by a script in the released code. The code, configuration libraries, per-experiment CSV files, and scripts that produce every table and figure of this article are available at https://github.com/adeliusa486/-lira-sdn-energy.

CRediT authorship contribution statement

Adeel Ahmad: Conceptualization, Methodology, Software, Investigation, Data curation, Visualization, Writing – original draft. **Toqeer Ali Syed:** Supervision, Validation, Writing – review & editing. **Ali Akarma:** Formal analysis, Validation, Writing – review & editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] M. Gupta, S. Singh, Greening of the internet, in: Proc. ACM SIGCOMM, 2003, pp. 19–26. doi:10.1145/863955.863959.

[2] A. P. Bianzino, C. Chaudet, D. Rossi, J.-L. Rougier, A survey of green networking research, IEEE Communications Surveys & Tutorials 14 (1) (2012) 3–20. doi:10.1109/SURV.2011.113010.00106.

[3] R. Bolla, R. Bruschi, F. Davoli, F. Cucchietti, Energy efficiency in the future internet: A survey of existing approaches and trends in energy-aware fixed network infrastructures, IEEE Communications Surveys & Tutorials 13 (2) (2011) 223–244. doi:10.1109/SURV.2011.071410.00073.

[4] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, J. Turner, OpenFlow: Enabling innovation in campus networks, ACM SIGCOMM Computer Communication Review 38 (2) (2008) 69–74. doi:10.1145/1355734.1355746.

[5] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, S. Uhlig, Software-defined networking: A comprehensive survey, Proceedings of the IEEE 103 (1) (2015) 14–76. doi:10.1109/JPROC.2014.2371999.

[6] L. Chiaraviglio, M. Mellia, F. Neri, Minimizing ISP network energy cost: Formulation and solutions, IEEE/ACM Transactions on Networking 20 (2) (2012) 463–476. doi:10.1109/TNET.2011.2161487.

[7] J. Gómez-delaHiz, M. Jiménez-Lázaro, J. L. Herrera, M. F. Zhani, J. Galán-Jiménez, On optimizing energy efficiency in SDN networks through ML-driven configuration, in: Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2025. doi:10.1109/NOMS57970.2025.11073657.

[8] J. Gómez-delaHiz, M. Jiménez-Lázaro, J. L. Herrera, M. F. Zhani, J. Galán-Jiménez, AI-driven scalable framework for energy-efficient SDN configurations, IEEE Transactions on Green Communications and Networking (2026). doi:10.1109/TGCN.2026.3724271.

[9] Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, M. Yu, Teal: Learning-accelerated optimization of WAN traffic engineering, in: Proc. ACM SIGCOMM, 2023. doi:10.1145/3603269.3604857.

[10] Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, A. Tamar, DOTE: Rethinking (predictive) WAN traffic engineering, in: Proc. USENIX NSDI, 2023. URL https://www.usenix.org/conference/nsdi23/presentation/perry

[11] F. Abuzaid, S. Kandula, B. Arzani, I. Menache, M. Zaharia, P. Bailis, Contracting wide-area network topologies to solve flow problems quickly, in: Proc. USENIX NSDI, 2021.
URL https://www.usenix.org/conference/nsdi21/presentation/abuzaid

[12] P. Almasan, S. Xiao, X. Cheng, X. Shi, P. Barlet-Ros, A. Cabellos-Aparicio, ENERO: Efficient real-time WAN routing optimization with deep reinforcement learning, Computer Networks 214 (2022) 109166. doi:10.1016/j.comnet.2022.109166.

[13] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: A methodological tour d'horizon, European Journal of Operational Research 290 (2) (2021) 405–421. doi:10.1016/j.ejor.2020.07.063.

[14] M. Reitblatt, N. Foster, J. Rexford, C. Schlesinger, D. Walker, Abstractions for network update, in: Proc. ACM SIGCOMM, 2012, pp. 323–334. doi:10.1145/2342356.2342427.

[15] X. Jin, H. H. Liu, R. Gandhi, S. Kandula, R. Mahajan, M. Zhang, J. Rexford, R. Wattenhofer, Dynamic scheduling of network updates, in: Proc. ACM SIGCOMM, 2014, pp. 539–550. doi:10.1145/2619239.2626307.

[16] L. Chiaraviglio, P. Wiatr, P. Monti, J. Chen, J. Lorincz, F. Idzikowski, M. Listanti, L. Wosinska, Is green networking beneficial in terms of device lifetime?, IEEE Communications Magazine 53 (5) (2015) 232–240. doi:10.1109/MCOM.2015.7105670.

[17] L. Chiaraviglio, et al., Impact of energy-efficient techniques on a device lifetime, in: Proc. IEEE Online Conference on Green Communications (OnlineGreenComm), 2014. doi:10.1109/OnlineGreenCom.2014.7114426.

[18] S. Ross, G. J. Gordon, J. A. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: Proc. AISTATS, 2011, pp. 627–635.
URL https://proceedings.mlr.press/v15/ross11a.html

[19] M. Zhang, C. Yi, B. Liu, B. Zhang, GreenTE: Power-aware traffic engineering, in: Proc. IEEE Int. Conf. Network Protocols (ICNP), 2010, pp. 21–30. doi:10.1109/ICNP.2010.5762751.

[20] B. Heller, S. Seetharaman, P. Mahadevan, Y. Yiakoumis, P. Sharma, S. Banerjee, N. McKeown, ElasticTree: Saving energy in data center networks, in: Proc. USENIX NSDI, 2010.
URL https://www.usenix.org/conference/nsdi10-0/elastictree-saving-energy-data-center-networks

[21] L. Röllin, R. Jacob, L. Vanbever, A sleep study for ISP networks: Evaluating link sleeping on real world data, ACM SIGEnergy Energy Informatics Review 4 (2024) 51–58. doi:10.1145/3727200.3727209.

[22] Y. Wang, Y. Li, T. Wang, G. Liu, Towards an energy-efficient data center network based on deep reinforcement learning, Computer Networks 210 (2022) 108939. doi:10.1016/j.comnet.2022.108939.

[23] P. Fröhlich, E. Gelenbe, M. P. Nowak, Reinforcement learning and energy-aware routing, in: Proc. ACM FlexNets Workshop, 2021. doi:10.1145/3472735.3473390.

[24] M. Ye, J. Zhang, Z. Guo, H. J. Chao, FlexDATE: Flexible and disturbance-aware traffic engineering with reinforcement learning in software-defined networks, IEEE/ACM Transactions on Networking 31 (4) (2023) 1433–1448. doi:10.1109/TNET.2022.3217083.

[25] A. Borodin, N. Linial, M. E. Saks, An optimal on-line algorithm for metrical task system, Journal of the ACM 39 (4) (1992) 745–763. doi:10.1145/146585.146588.

[26] A. Antoniadis, C. Coester, M. Eliáš, A. Polak, B. Simon, Online metric algorithms with untrusted predictions, ACM Transactions on Algorithms 19 (2) (2023) 19:1–19:34. doi:10.1145/3582689.

[27] N. Christianson, J. Shen, A. Wierman, Optimal robustness-consistency tradeoffs for learning-augmented metrical task systems, in: Proc. 26th Int. Conf. Artificial Intelligence and Statistics (AISTATS), Vol. 206 of PMLR, 2023, pp. 9377–9399.

[28] J. C. C. Restrepo, C. G. Gruber, C. M. Machuca, Energy profile aware routing, in: Proc. IEEE ICC Workshops, 2009. doi:10.1109/ICCW.2009.5208041.

[29] P. Mahadevan, P. Sharma, S. Banerjee, P. Ranganathan, A power benchmarking framework for network devices, in: Proc. IFIP Networking, Vol. 5550 of LNCS, 2009, pp. 795–808. doi:10.1007/978-3-642-01399-7_62.

[30] A. Vishwanath, K. Hinton, R. W. A. Ayre, R. S. Tucker, Modeling energy consumption in high-capacity routers and switches, IEEE Journal on Selected Areas in Communications 32 (8) (2014) 1524–1532. doi:10.1109/JSAC.2014.2335312.

[31] Y. Zhang, M. Roughan, N. Duffield, A. Greenberg, Fast accurate computation of large-scale IP traffic matrices from link loads, in: Proc. ACM SIGMETRICS, 2003, pp. 206–217. doi:10.1145/781027.781053.

[32] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Transactions on Evolutionary Computation 6 (2) (2002) 182–197. doi:10.1109/4235.996017.

[33] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proc. ACM SIGKDD, 2016, pp. 785–794. doi:10.1145/2939672.2939785.

[34] G. Ke, et al., LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems (NeurIPS), 2017.
URL https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree

[35] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.

[36] S. Uhlig, B. Quoitin, J. Lepropre, S. Balon, Providing public intradomain traffic matrices to the research community, ACM SIGCOMM Computer Communication Review 36 (1) (2006) 83–86. doi:10.1145/1111322.1111341.

[37] S. Orlowski, R. Wessäly, M. Pióro, A. Tomaszewski, SNDlib 1.0—survivable network design library, Networks 55 (3) (2010) 276–286. doi:10.1002/net.20371.

[38] Q. Huangfu, J. A. J. Hall, Parallelizing the dual revised simplex method, Mathematical Programming Computation 10 (1) (2018) 119–142. doi:10.1007/s12532-017-0130-5.

[39] T. Lykouris, S. Vassilvitskii, Competitive caching with machine learned advice, Journal of the ACM 68 (4) (2021) 24:1–24:25. doi:10.1145/3447579.